Tools Setup

In [1]:
```
import pathlib, os, sys, operator, re, datetime
from functools import reduce
import numpy as np
import tensorflow as tf
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow import keras
from keras import Model
import tensorflow_datasets as tfds
from tiny_imagenet import TinyImagenetDataset

# Enable or disable GPU
# To fully disable it, we need to hide all GPU devices from Tensorflow
# Make sure GPU is disabled for this inference part of the lab
ENABLE_GPU = True
# tf.debugging.set_log_device_placement(True)

if not ENABLE_GPU:
    tf.config.set_visible_devices([], 'GPU')

# Print Python and TF version, and where we are running
print(f'Running on Python Version: {sys.version}')
print(f'Using Tensorflow Version: {tf. __version__}')
if not tf.config.experimental.list_physical_devices("GPU"):
    print('Running on CPU')
else:
    print(f'Using GPU at: {tf.test.gpu_device_name()} (of {len(tf.config.experimental.list_physical_devices("GPU"))} available)')
```

```
2026-10-07 16:44:22.301328: I tensorflow/core/util/port.cc:113] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point
round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2026-10-07 16:44:22.317150: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:479] Unable to register cuFFT factory: Attempting to register factory for plugin
cuFFT when one has already been registered
2026-10-07 16:44:22.334403: E external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:10575] Unable to register cuDNN factory: Attempting to register factory for plugi
n cuDNN when one has already been registered
2026-10-07 16:44:22.334428: E external/local_xla/xla/stream_executor/cuda/cuda_blas.cc:1442] Unable to register cuBLAS factory: Attempting to register factory for plug
in cuBLAS when one has already been registered
2026-10-07 16:44:22.345663: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-c
ritical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
/home/jwk0425/jwk_personal_lab/lab1/lab1_venv/lib64/python3.9/site-packages/google/api_core/_python_version_support.py:246: FutureWarning: You are using a non-supporte
d Python version (3.9.25). Google will not post any further updates to google.api_core supporting this Python version. Please upgrade to the latest Python version, or
at least Python 3.11, and then update google.api_core.
  warnings.warn(message, FutureWarning)
/home/jwk0425/jwk_personal_lab/lab1/lab1_venv/lib64/python3.9/site-packages/google/auth/__init__.py:54: FutureWarning: You are using a Python version 3.9 past its end
of life. Google will update google-auth with critical bug fixes on a best-effort basis, but not with any other fixes or features. Please upgrade your Python version, a
nd then update google-auth.
  warnings.warn(eol_message.format("3.9"), FutureWarning)
2026-10-07 16:44:23.763605: W tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38] TF-TRT Warning: Could not find TensorRT
Running on Python Version: 3.9.25 (main, Apr 17 2026, 00:00:00)
[GCC 11.5.0 20240719 (Red Hat 11.5.0-14)]
Using Tensorflow Version: 2.16.2
Using GPU at: /device:GPU:0 (of 1 available)
```

```
2026-10-07 16:44:27.213981: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there
must be at least one NUMA node, so returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355
2026-10-07 16:44:27.239378: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there
must be at least one NUMA node, so returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355
2026-10-07 16:44:27.239514: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there
must be at least one NUMA node, so returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355
2026-10-07 16:44:27.240333: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there
must be at least one NUMA node, so returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355
2026-10-07 16:44:27.240427: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there
must be at least one NUMA node, so returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355
2026-10-07 16:44:27.240484: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there
must be at least one NUMA node, so returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355
2026-10-07 16:44:27.296851: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there
must be at least one NUMA node, so returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355
2026-10-07 16:44:27.296949: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there
must be at least one NUMA node, so returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355
2026-10-07 16:44:27.297019: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there
must be at least one NUMA node, so returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355
2026-10-07 16:44:27.297912: I tensorflow/core/common_runtime/gpu/gpu_device.cc:1928] Created device /device:GPU:0 with 5525 MB memory:  -> device: 0, name: NVIDIA A4
0-8Q, pci bus id: 0000:00:10.0, compute capability: 8.6
```

Dataset Inspection

In [2]:

```python
# This cell imports our dataset.

# Original Source: https://github.com/ksachdeva/tiny-imagenet-tfds
# Class Version Source: https://github.com/duweisu/tiny-imagenet-tfds
# Setup our dataset
# ----------------------------------------------------------

tiny_imagenet_builder = TinyImagenetDataset()

# this call (download_and_prepare) will trigger the download of the dataset
# and preparation (conversion to tfrecords)
#
# This will be done only once and on next usage tfds will
# use the cached version on your host.
tiny_imagenet_builder.download_and_prepare(download_dir="~/tensorflow-datasets/downloads")

# class_names = tiny_imagenet_builder.info.features['label'].names
ds = tiny_imagenet_builder.as_dataset()
ds_train, ds_val = ds["train"], ds["validation"]
assert(isinstance(ds_train, tf.data.Dataset))
assert(isinstance(ds_val, tf.data.Dataset))

# Training Dataset
ds_train = ds_train.shuffle(1024).prefetch(tf.data.AUTOTUNE)

# Validation Dataset
ds_val = ds_val.shuffle(1024).prefetch(tf.data.AUTOTUNE)

# Dataset metadata
ds_info = tiny_imagenet_builder.info
```

```
WARNING:absl:You use TensorFlow DType <dtype: 'string'> in tfds.features This will soon be deprecated in favor of NumPy DTypes. In the meantime it was converted to obj
ect.
2026-10-07 16:44:29.216803: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there
must be at least one NUMA node, so returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355
2026-10-07 16:44:29.216955: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there
must be at least one NUMA node, so returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355
2026-10-07 16:44:29.217015: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there
must be at least one NUMA node, so returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355
2026-10-07 16:44:29.217109: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there
must be at least one NUMA node, so returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355
2026-10-07 16:44:29.217178: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:998] successful NUMA node read from SysFS had negative value (-1), but there
must be at least one NUMA node, so returning NUMA node zero. See more at https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355
2026-10-07 16:44:29.217225: I tensorflow/core/common_runtime/gpu/gpu_device.cc:1928] Created device /job:localhost/replica:0/task:0/device:GPU:0 with 5525 MB memory:
-> device: 0, name: NVIDIA A40-8Q, pci bus id: 0000:00:10.0, compute capability: 8.6
```

In [3]:
```python
# We need to read the "human readable" labels so we can translate with the numeric values
# Read the labels file (words.txt)
with open(os.path.abspath('wnids.txt'), 'r') as f:
    wnids = [x.strip() for x in f]

# Map wnids to integer labels
wnid_to_label = {wnid: i for i, wnid in enumerate(wnids)}
label_to_wnid = {v: k for k, v in wnid_to_label.items()}

# Use words.txt to get names for each class
with open(os.path.abspath('words.txt'), 'r') as f:
    wnid_to_words = dict(line.split('\t') for line in f)
    for wnid, words in wnid_to_words.items():
        wnid_to_words[wnid] = [w.strip() for w in words.split(',')]

class_names = [str(wnid_to_words[wnid]) for wnid in wnids]
```

In [4]:
```python
# Helper function to get the label name
def img_class(img_data, idx=None):
    image, label, id, label_name = img_data["image"], img_data["label"], img_data["id"], img_data["metadata"]['label_name']
    # Handle batches of images correctly
    if idx != None:
        image, label, id, label_name = img_data["image"][idx], img_data["label"][idx], img_data["id"][idx], img_data["metadata"]['label_name'][idx]

    return f"{label_name} (class index: {label} - id: {id})"


# Helper function to show basic info about an image
def img_info(img, idx=None, display=True, title_apend=""):
    image = img['image']

    # Print the class
    class_str = img_class(img, idx)
    print(f"Label: {class_str}")

    # Display the image
    if display:
        plt.figure()
        plt.title(title_apend + class_str)
        # Handle batches correctly
        if image.shape.ndims > 3:
            plt.imshow(image.numpy().reshape(64, 64, 3))
        else:
            plt.imshow(image.numpy())
```

In [5]:
```python
# Print the dataset types and info
```

```
print("--- Train & Validation dataset info ---")
print(f"Train: {ds_train}")
print(f"Validation: {ds_val}")
# print(f"Dataset Info: {ds_info}") # Uncomment to print Dataset info

print("\n--- Show an example image ---")
for example in ds_val.take(1):
    img_info(example)

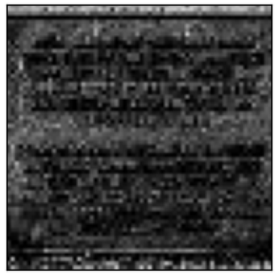
print("\n Show some other examples")
tfds.show_examples(ds_val, ds_info, rows=3, cols=3)
```

```
--- Train & Validation dataset info ---
Train: <_PrefetchDataset element_spec={'id': TensorSpec(shape=(), dtype=tf.string, name=None), 'image': TensorSpec(shape=(64, 64, 3), dtype=tf.uint8, name=None), 'labe
l': TensorSpec(shape=(), dtype=tf.int64, name=None), 'metadata': {'label_name': TensorSpec(shape=(), dtype=tf.string, name=None)}}>
Validation: <_PrefetchDataset element_spec={'id': TensorSpec(shape=(), dtype=tf.string, name=None), 'image': TensorSpec(shape=(64, 64, 3), dtype=tf.uint8, name=None),
'label': TensorSpec(shape=(), dtype=tf.int64, name=None), 'metadata': {'label_name': TensorSpec(shape=(), dtype=tf.string, name=None)}}>

--- Show an example image ---
Label: b'German shepherd, German shepherd dog, German police dog, alsatian' (class index: 11 - id: b'n02106662')

 Show some other examples
2026-10-07 16:07:20.070595: W tensorflow/core/kernels/data/cache_dataset_ops.cc:858] The calling iterator did not fully read the dataset being cached. In order to avoi
d unexpected truncation of the dataset, the partially cached contents of the dataset  will be discarded. This can happen if you have an input pipeline similar to `data
set.cache().take(k).repeat()`. You should use `dataset.take(k).cache().repeat()` instead.
2026-10-07 16:07:20.104408: W tensorflow/core/framework/local_rendezvous.cc:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
2026-10-07 16:07:20.297195: W tensorflow/core/kernels/data/cache_dataset_ops.cc:858] The calling iterator did not fully read the dataset being cached. In order to avoi
d unexpected truncation of the dataset, the partially cached contents of the dataset  will be discarded. This can happen if you have an input pipeline similar to `data
set.cache().take(k).repeat()`. You should use `dataset.take(k).cache().repeat()` instead.
2026-10-07 16:07:20.310804: W tensorflow/core/framework/local_rendezvous.cc:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
```




155 (155)


8 (8)


26 (26)


87 (87)


86 (86)


96 (96)


59 (59)


69 (69)


101 (101)

Out[5]:



In [6]:
```python
# TODO: Print and visualize three inputs from the validation set
#     : Print the stroage data type
#     : Print and note the dimensions of each image
#     : Print the memory required to store each image

# Sample Images
sample_imgs = []
for index, img_data in enumerate(ds["validation"].take(3)):
    sample_imgs.append(img_data)
    image, label, id, label_name = img_data["image"], img_data["label"], img_data["id"], img_data["metadata"]['label_name']

    print(f'\n--- Image {index} ---')
    # TODO: Your Code Here
    # See example usage: https://github.com/duweisu/tiny-imagenet-tfds
    img_info(img_data)
    print(f'Image id: {id.numpy().decode()}')
    print(f'Storage data type: {image.dtype.name}')
```

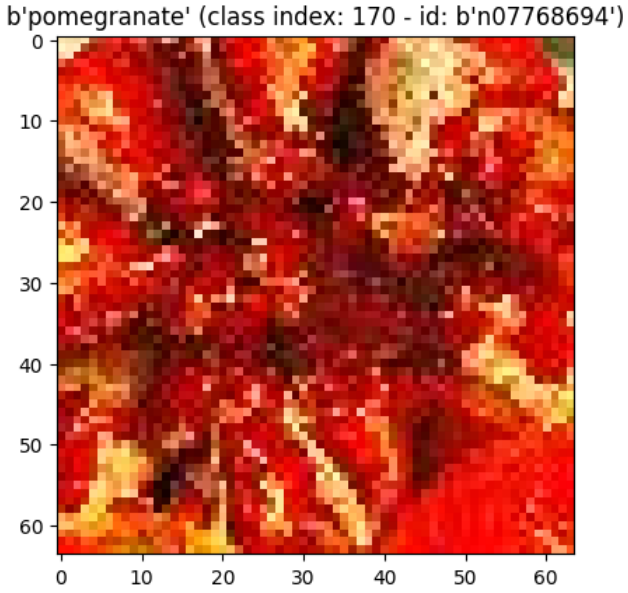
```
    print(f'Dimensions (H, W, C): {tuple(image.shape)}')
    print(f'Memory required: {image.numpy().nbytes} bytes')
```

```
--- Image 0 ---
Label: b'pomegranate' (class index: 170 - id: b'n07768694')
Image id: n07768694
Storage data type: uint8
Dimensions (H, W, C): (64, 64, 3)
Memory required: 12288 bytes

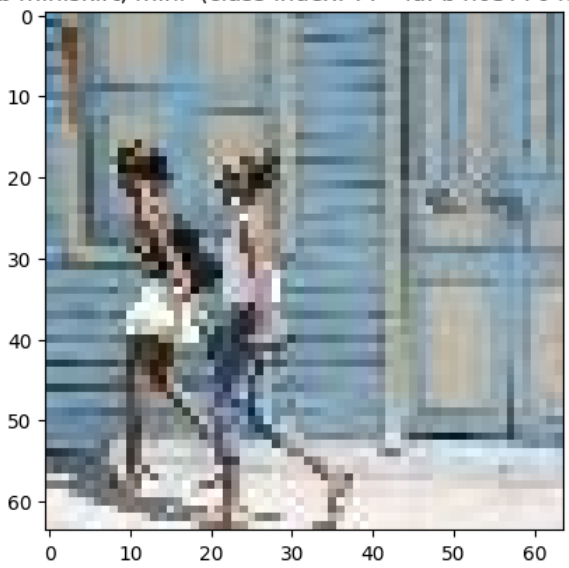
--- Image 1 ---
Label: b'miniskirt, mini' (class index: 77 - id: b'n03770439')
Image id: n03770439
Storage data type: uint8
Dimensions (H, W, C): (64, 64, 3)
Memory required: 12288 bytes

--- Image 2 ---
Label: b'German shepherd, German shepherd dog, German police dog, alsatian' (class index: 11 - id: b'n02106662')
Image id: n02106662
Storage data type: uint8
Dimensions (H, W, C): (64, 64, 3)
Memory required: 12288 bytes
```

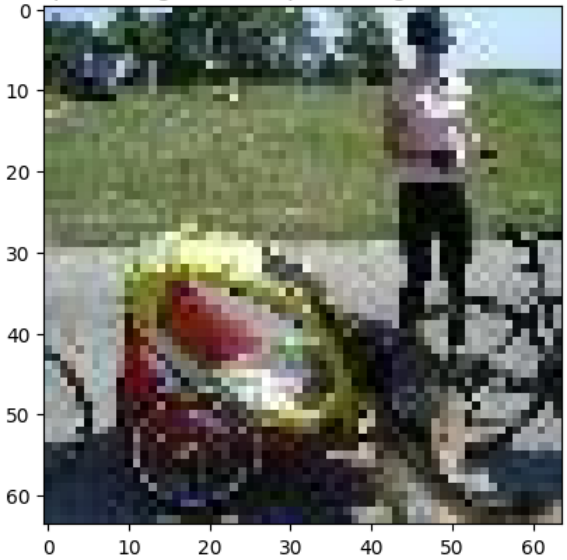
```
2026-10-07 16:07:21.288134: W tensorflow/core/kernels/data/cache_dataset_ops.cc:858] The calling iterator did not fully read the dataset being cached. In order to avoi
d unexpected truncation of the dataset, the partially cached contents of the dataset  will be discarded. This can happen if you have an input pipeline similar to `data
set.cache().take(k).repeat()`. You should use `dataset.take(k).cache().repeat()` instead.
2026-10-07 16:07:21.297077: W tensorflow/core/framework/local_rendezvous.cc:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
```



b'miniskirt, mini' (class index: 77 - id: b'n03770439')



b'German shepherd, German shepherd dog, German police dog, alsatian' (class index: 11 - id: b'n02106662')



In [7]:
```
# TODO: Export each of the three inputs to a binary file which will be used to load the images into C++ later
# NOTE: First flatten the array (ex: 4D --> 1D). So 64*64*3 = 12288 element 1D array

# Make a directory for our image data
img_dir = os.path.abspath('img_data')
pathlib.Path(img_dir).mkdir(exist_ok=True)

# Create a metadata file
metadata_file = open(os.path.join(img_dir, f'metadata.txt'), 'w')
```

```
metadata_file.write(f'Number\t\tDims\t\tClass Data\n')

# Export each image
for index, img_data in enumerate(sample_imgs):
    img_file = open(os.path.join(img_dir, f'image_{index}.bin'), 'wb')

    # TODO: Your Code Here
    # Raw uint8 bytes, flattened in (H, W, C) order
    img_array = img_data["image"].numpy().flatten()
    img_array.tofile(img_file)
    img_file.close()
    assert os.path.getsize(img_file.name) == 64 * 64 * 3

    metadata_file.write(f'{index}\t\t{tuple(img_data["image"].shape)}\t\t{img_class(img_data)}\n')
    print(f'Exported {img_file.name}: {img_array.size} elements, {img_array.dtype}')

metadata_file.close()
print(open(metadata_file.name).read())
```

```
Exported /home/jwk0425/jwk_personal_lab/lab1/img_data/image_0.bin: 12288 elements, uint8
Exported /home/jwk0425/jwk_personal_lab/lab1/img_data/image_1.bin: 12288 elements, uint8
Exported /home/jwk0425/jwk_personal_lab/lab1/img_data/image_2.bin: 12288 elements, uint8
Number          Dims            Class Data
0               (64, 64, 3)             b'pomegranate' (class index: 170 - id: b'n07768694')
1               (64, 64, 3)             b'miniskirt, mini' (class index: 77 - id: b'n03770439')
2               (64, 64, 3)             b'German shepherd, German shepherd dog, German police dog, alsatian' (class index: 11 - id: b'n02106662')
```

Model Loading and Inference

In [8]:
```
# TODO: Load the model
# Now we will load the H5 model! Please make sure the h5 model file is present in the below directory.
# You can download this from the Canvas Page and place it in the same directory as this notebook.

# model_path = os.path.abspath(""/home/<NETID>/path/to/your/lab1/CNN_TinyImageNet.h5)" # Uncomment this to use a non-relative path
model_path = os.path.abspath("CNN_TinyImageNet.h5")

# TODO: Your Code Here
model = tf.keras.models.load_model(model_path)
model.summary()
```

```
/home/jwk0425/jwk_personal_lab/lab1/lab1_venv/lib64/python3.9/site-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/
`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/home/jwk0425/jwk_personal_lab/lab1/lab1_venv/lib64/python3.9/site-packages/keras/src/optimizers/base_optimizer.py:86: UserWarning: Argument `decay` is no longer suppo
rted and will be ignored.
  warnings.warn(
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_metrics` will be empty until you train or evaluate the model.
WARNING:absl:Error in loading the saved optimizer state. As a result, your model is starting with a freshly initialized optimizer.
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 60, 60, 32)	2,432
conv2d_1 (Conv2D)	(None, 56, 56, 32)	25,632
max_pooling2d (MaxPooling2D)	(None, 28, 28, 32)	0
conv2d_2 (Conv2D)	(None, 26, 26, 64)	18,496
conv2d_3 (Conv2D)	(None, 24, 24, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 12, 12, 64)	0
conv2d_4 (Conv2D)	(None, 10, 10, 64)	36,928
conv2d_5 (Conv2D)	(None, 8, 8, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	0
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 256)	524,544
dense_1 (Dense)	(None, 200)	51,400

Total params: 770,218 (2.94 MB)
Trainable params: 770,216 (2.94 MB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 2 (12.00 B)

In [9]:
```
# Running infrence on our model
# We can run an infrence of our model by doing the following (we are doing batches of 1 here)
for example in ds_train.batch(1).take(1):
    img_info(example)

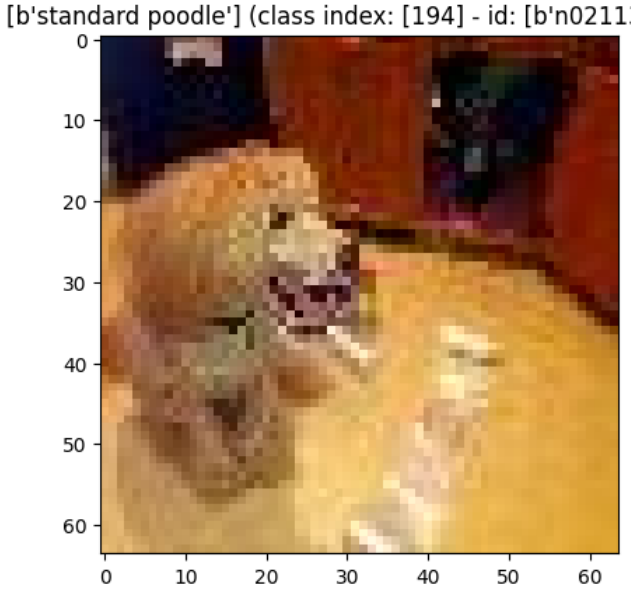
    # Make a prediction
    pred = model.predict(tf.cast(example["image"], tf.float32)/255.0)
    # print(f'Raw 200 Class Weighted Prediction:\n{pred}') # Uncomment to see the raw prediction

    # What is out best guess?
    best_guess = tf.math.argmax(pred, axis=1).numpy() # Our output is 200 weighted value, we want the most likely
    print(f'Best Guess [class index]: {class_names[best_guess[0]]} [{best_guess[0]}]')
    print(f'Best Guess Confidence (percent / 1.0): {pred[0][best_guess]}')

    # What are our top 15 guesses?
    top_15 = tf.math.top_k(pred, k=15)
    print(f'Top 15 Guesses (class index): {[f"{class_names[idx]} [{idx}]" for idx in top_15.indices[0]]}')
    print(f'Top 15 Guesses Confidence (percent / 1.0): {top_15.values}')
```

```
2026-10-07 16:07:22.134099: W tensorflow/core/kernels/data/cache_dataset_ops.cc:858] The calling iterator did not fully read the dataset being cached. In order to avoi
d unexpected truncation of the dataset, the partially cached contents of the dataset  will be discarded. This can happen if you have an input pipeline similar to `data
set.cache().take(k).repeat()`. You should use `dataset.take(k).cache().repeat()` instead.
```

```
Label: [b'standard poodle'] (class index: [194] - id: [b'n02113799'])
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 115ms/step
Best Guess [class index]: ['Persian cat'] [131]
Best Guess Confidence (percent / 1.0): [0.07200943]
Top 15 Guesses (class index): ["['Persian cat'] [131]", "['ice cream', 'icecream'] [28]", "['mashed potato'] [167]", "['sandal'] [97]", "['American lobster', 'Northern
lobster', 'Maine lobster', 'Homarus americanus'] [151]", "['computer keyboard', 'keypad'] [26]", "['plate'] [143]", '["potter\'s wheel"] [96]', "['wooden spoon'] [12
6]", "['potpie'] [23]", "['bathtub', 'bathing tub', 'bath', 'tub'] [179]", "['orange'] [149]", "['nail'] [29]", "['drumstick'] [153]", "['scorpion'] [125]"]
Top 15 Guesses Confidence (percent / 1.0): [[0.07200943 0.07150489 0.06922593 0.03841855 0.03610497 0.03452906
  0.02528321 0.02417605 0.02141708 0.0207166  0.01927122 0.01868135
  0.0185076  0.01779644 0.0176456 ]]
2026-10-07 16:07:22.389699: W tensorflow/core/framework/local_rendezvous.cc:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
```



In [10]:
```python
# TODO: Run infrence for our previous 3 sample images

# TODO: Your Code Here
# Same preprocessing as the example above: uint8 [0, 255] -> float32 [0, 1], one batch of 3
sample_batch = tf.stack([tf.cast(img_data["image"], tf.float32) / 255.0 for img_data in sample_imgs])
sample_preds = model.predict(sample_batch, verbose=0)

def short_name(idx):
    # First (representative) name of the class, e.g. 'cash machine'
    return wnid_to_words[wnids[idx]][0]

print(f'{"Image":<6}{"Best Guess [index]":<28}{"Confidence":<12}{"Actual Class [index]":<28}Correct')
for index, (img_data, pred) in enumerate(zip(sample_imgs, sample_preds)):
    best_guess = int(np.argmax(pred))
    actual = int(img_data["label"].numpy())
    print(f'{index:<6}{f"{short_name(best_guess)} [{best_guess}]":<28}{pred[best_guess]:<12.4f}'
          f'{f"{short_name(actual)} [{actual}]":<28}{best_guess == actual}')
```

```
Image Best Guess [index]          Confidence  Actual Class [index]        Correct
0     pomegranate [170]           0.7293      pomegranate [170]           True
1     cash machine [106]          0.1033      miniskirt [77]              False
2     rugby ball [128]            0.2106      German shepherd [11]        False
```

In [11]:
```python
# TODO: Calculate the Top-1, Top-5, and Top-10 Accuracy of the validation dataset
```

```
total = acc_top1 = acc_top5 = acc_top10 = 0

# TODO: Your Code Here
# Batched inference over the first 10,000 validation images (the whole validation split)
for batch in ds_val.take(10000).batch(100):
    pred = model.predict(tf.cast(batch["image"], tf.float32) / 255.0, verbose=0)
    labels = batch["label"].numpy()

    # hits[i, k] is True when the k-th most likely class of image i is the ground truth
    top_10 = tf.math.top_k(pred, k=10).indices.numpy()
    hits = top_10 == labels[:, None]

    acc_top1 += int(hits[:, :1].sum())
    acc_top5 += int(hits[:, :5].sum())
    acc_top10 += int(hits.sum())
    total += len(labels)

print(f'Total Images: {total}')
print(f'Top-1 Accuracy: {acc_top1 / total * 100:.2f}%')
print(f'Top-5 Accuracy: {acc_top5 / total * 100:.2f}%')
print(f'Top-10 Accuracy: {acc_top10 / total * 100:.2f}%')
```

```
Total Images: 10000
Top-1 Accuracy: 24.30%
Top-5 Accuracy: 49.40%
Top-10 Accuracy: 61.74%
2026-10-07 16:07:40.432062: W tensorflow/core/framework/local_rendezvous.cc:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
```

In [12]:

```
# TODO: Print all of the possible classes of the dataset

# TODO: Your Code Here
print('--- All classes ---')
for index, name in enumerate(class_names):
    print(f'{index}: {name}')

train_count = ds_info.splits['train'].num_examples
val_count = ds_info.splits['validation'].num_examples
print(f'\nNumber of classes: {len(class_names)}')
print(f'Train images: {train_count} ({train_count // len(class_names)} per class)')
print(f'Validation images: {val_count} ({val_count // len(class_names)} per class)')
print(f'Total images: {train_count + val_count}')
```

```
--- All classes ---
0: ['Egyptian cat']
1: ['reel']
2: ['volleyball']
3: ['rocking chair', 'rocker']
4: ['lemon']
5: ['bullfrog', 'Rana catesbeiana']
6: ['basketball']
7: ['cliff', 'drop', 'drop-off']
8: ['espresso']
9: ['plunger', "plumber's helper"]
10: ['parking meter']
11: ['German shepherd', 'German shepherd dog', 'German police dog', 'alsatian']
12: ['dining table', 'board']
13: ['monarch', 'monarch butterfly', 'milkweed butterfly', 'Danaus plexippus']
14: ['brown bear', 'bruin', 'Ursus arctos']
15: ['school bus']
16: ['pizza', 'pizza pie']
17: ['guinea pig', 'Cavia cobaya']
18: ['umbrella']
19: ['organ', 'pipe organ']
20: ['oboe', 'hautboy', 'hautbois']
21: ['maypole']
22: ['goldfish', 'Carassius auratus']
23: ['potpie']
24: ['hourglass']
25: ['seashore', 'coast', 'seacoast', 'sea-coast']
26: ['computer keyboard', 'keypad']
27: ['Arabian camel', 'dromedary', 'Camelus dromedarius']
28: ['ice cream', 'icecream']
29: ['nail']
30: ['space heater']
31: ['cardigan']
32: ['baboon']
33: ['snail']
34: ['coral reef']
35: ['albatross', 'mollymawk']
36: ['spider web', "spider's web"]
37: ['sea cucumber', 'holothurian']
38: ['backpack', 'back pack', 'knapsack', 'packsack', 'rucksack', 'haversack']
39: ['Labrador retriever']
40: ['pretzel']
41: ['king penguin', 'Aptenodytes patagonica']
42: ['sulphur butterfly', 'sulfur butterfly']
43: ['tarantula']
44: ['lesser panda', 'red panda', 'panda', 'bear cat', 'cat bear', 'Ailurus fulgens']
45: ['pop bottle', 'soda bottle']
46: ['banana']
47: ['sock']
48: ['cockroach', 'roach']
49: ['projectile', 'missile']
50: ['beer bottle']
51: ['mantis', 'mantid']
52: ['freight car']
53: ['guacamole']
54: ['remote control', 'remote']
55: ['European fire salamander', 'Salamandra salamandra']
56: ['lakeside', 'lakeshore']
57: ['chimpanzee', 'chimp', 'Pan troglodytes']
58: ['pay-phone', 'pay-station']
59: ['fur coat']
60: ['alp']
```

```
61: ['lampshade', 'lamp shade']
62: ['torch']
63: ['abacus']
64: ['moving van']
65: ['barrel', 'cask']
66: ['tabby', 'tabby cat']
67: ['goose']
68: ['koala', 'koala bear', 'kangaroo bear', 'native bear', 'Phascolarctos cinereus']
69: ['bullet train', 'bullet']
70: ['CD player']
71: ['teapot']
72: ['birdhouse']
73: ['gazelle']
74: ['academic gown', 'academic robe', "judge's robe"]
75: ['tractor']
76: ['ladybug', 'ladybeetle', 'lady beetle', 'ladybird', 'ladybird beetle']
77: ['miniskirt', 'mini']
78: ['golden retriever']
79: ['triumphal arch']
80: ['cannon']
81: ['neck brace']
82: ['sombrero']
83: ['gasmask', 'respirator', 'gas helmet']
84: ['candle', 'taper', 'wax light']
85: ['desk']
86: ['frying pan', 'frypan', 'skillet']
87: ['bee']
88: ['dam', 'dike', 'dyke']
89: ['spiny lobster', 'langouste', 'rock lobster', 'crawfish', 'crayfish', 'sea crawfish']
90: ['police van', 'police wagon', 'paddy wagon', 'patrol wagon', 'wagon', 'black Maria']
91: ['iPod']
92: ['punching bag', 'punch bag', 'punching ball', 'punchball']
93: ['beacon', 'lighthouse', 'beacon light', 'pharos']
94: ['jellyfish']
95: ['wok']
96: ["potter's wheel"]
97: ['sandal']
98: ['pill bottle']
99: ['butcher shop', 'meat market']
100: ['slug']
101: ['hog', 'pig', 'grunter', 'squealer', 'Sus scrofa']
102: ['cougar', 'puma', 'catamount', 'mountain lion', 'painter', 'panther', 'Felis concolor']
103: ['crane']
104: ['vestment']
105: ['dragonfly', 'darning needle', "devil's darning needle", 'sewing needle', 'snake feeder', 'snake doctor', 'mosquito hawk', 'skeeter hawk']
106: ['cash machine', 'cash dispenser', 'automated teller machine', 'automatic teller machine', 'automated teller', 'automatic teller', 'ATM']
107: ['mushroom']
108: ['jinrikisha', 'ricksha', 'rickshaw']
109: ['water tower']
110: ['chest']
111: ['snorkel']
112: ['sunglasses', 'dark glasses', 'shades']
113: ['fly']
114: ['limousine', 'limo']
115: ['black stork', 'Ciconia nigra']
116: ['dugong', 'Dugong dugon']
117: ['sports car', 'sport car']
118: ['water jug']
119: ['suspension bridge']
120: ['ox']
121: ['ice lolly', 'lolly', 'lollipop', 'popsicle']
122: ['turnstile']
```

```
123: ['Christmas stocking']
124: ['broom']
125: ['scorpion']
126: ['wooden spoon']
127: ['picket fence', 'paling']
128: ['rugby ball']
129: ['sewing machine']
130: ['steel arch bridge']
131: ['Persian cat']
132: ['refrigerator', 'icebox']
133: ['barn']
134: ['apron']
135: ['Yorkshire terrier']
136: ['swimming trunks', 'bathing trunks']
137: ['stopwatch', 'stop watch']
138: ['lawn mower', 'mower']
139: ['thatch', 'thatched roof']
140: ['fountain']
141: ['black widow', 'Latrodectus mactans']
142: ['bikini', 'two-piece']
143: ['plate']
144: ['teddy', 'teddy bear']
145: ['barbershop']
146: ['confectionery', 'confectionary', 'candy store']
147: ['beach wagon', 'station wagon', 'wagon', 'estate car', 'beach waggon', 'station waggon', 'waggon']
148: ['scoreboard']
149: ['orange']
150: ['flagpole', 'flagstaff']
151: ['American lobster', 'Northern lobster', 'Maine lobster', 'Homarus americanus']
152: ['trolleybus', 'trolley coach', 'trackless trolley']
153: ['drumstick']
154: ['dumbbell']
155: ['brass', 'memorial tablet', 'plaque']
156: ['bow tie', 'bow-tie', 'bowtie']
157: ['convertible']
158: ['bighorn', 'bighorn sheep', 'cimarron', 'Rocky Mountain bighorn', 'Rocky Mountain sheep', 'Ovis canadensis']
159: ['orangutan', 'orang', 'orangutang', 'Pongo pygmaeus']
160: ['American alligator', 'Alligator mississipiensis']
161: ['centipede']
162: ['syringe']
163: ['go-kart']
164: ['brain coral']
165: ['sea slug', 'nudibranch']
166: ['cliff dwelling']
167: ['mashed potato']
168: ['viaduct']
169: ['military uniform']
170: ['pomegranate']
171: ['chain']
172: ['kimono']
173: ['comic book']
174: ['trilobite']
175: ['bison']
176: ['pole']
177: ['boa constrictor', 'Constrictor constrictor']
178: ['poncho']
179: ['bathtub', 'bathing tub', 'bath', 'tub']
180: ['grasshopper', 'hopper']
181: ['walking stick', 'walkingstick', 'stick insect']
182: ['Chihuahua']
183: ['tailed frog', 'bell toad', 'ribbed toad', 'tailed toad', 'Ascaphus trui']
184: ['lion', 'king of beasts', 'Panthera leo']
```

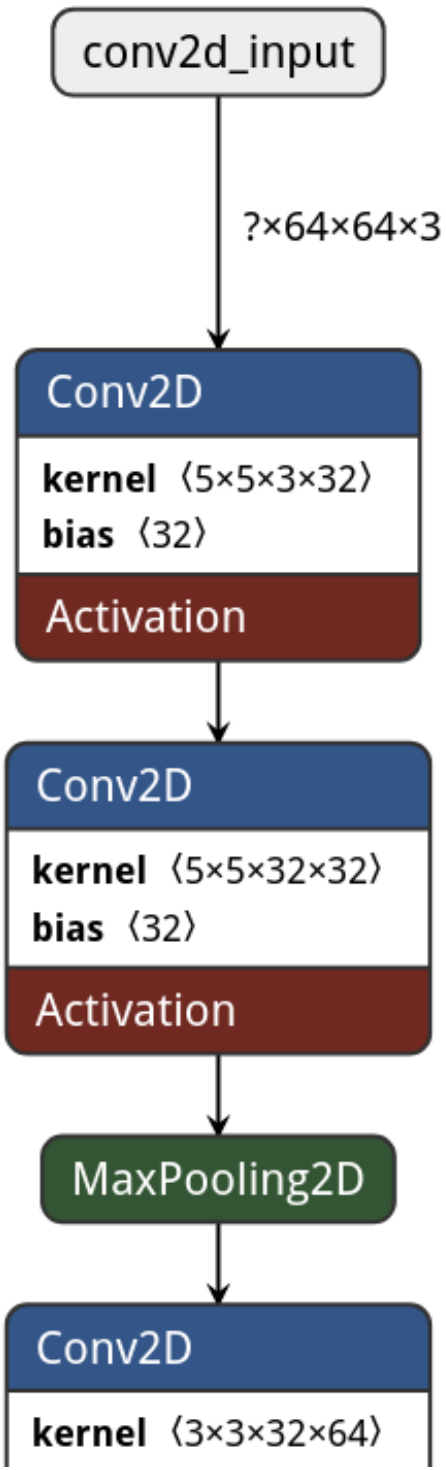
```
185: ['altar']
186: ['obelisk']
187: ['beaker']
188: ['bell pepper']
189: ['bannister', 'banister', 'balustrade', 'balusters', 'handrail']
190: ['bucket', 'pail']
191: ['magnetic compass']
192: ['meat loaf', 'meatloaf']
193: ['gondola']
194: ['standard poodle']
195: ['acorn']
196: ['lifeboat']
197: ['binoculars', 'field glasses', 'opera glasses']
198: ['cauliflower']
199: ['African elephant', 'Loxodonta africana']

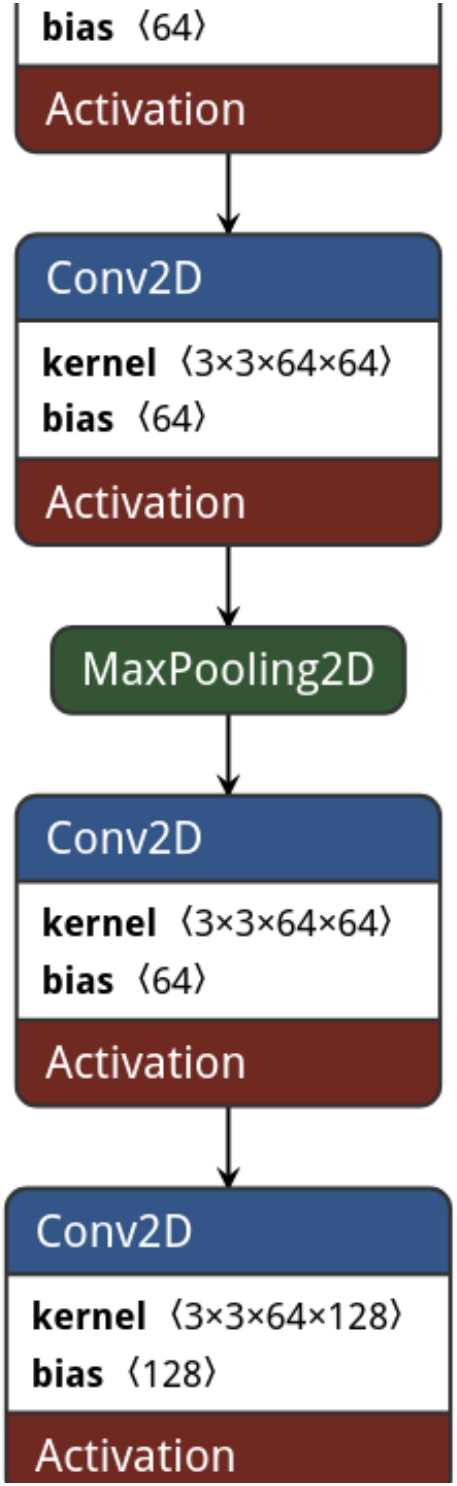
Number of classes: 200
Train images: 100000 (500 per class)
Validation images: 10000 (50 per class)
Total images: 110000
```

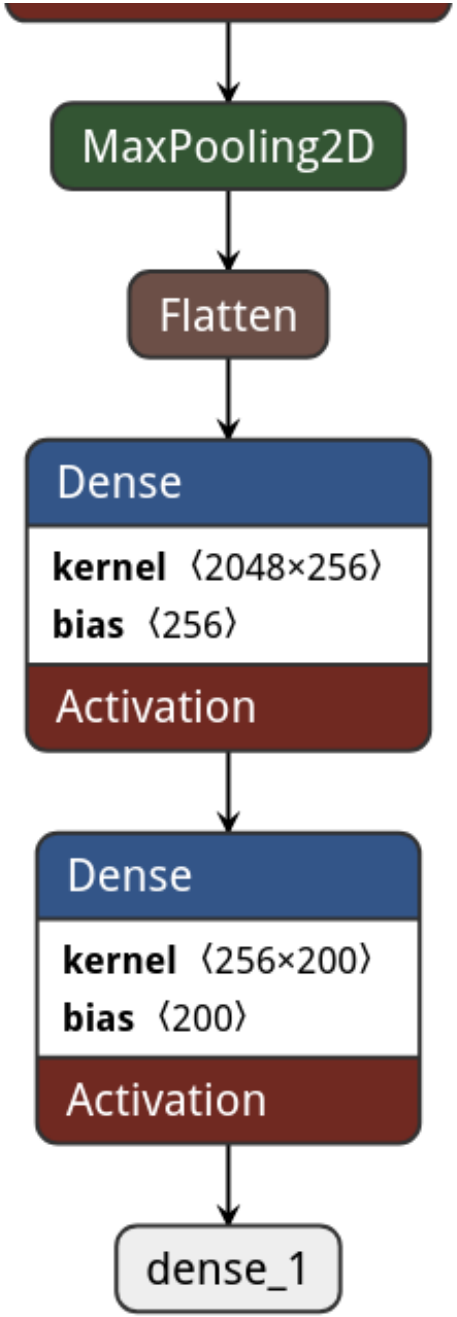
Model Exploration

In [13]:
```
# TODO: Visualize the model in Netron (https://netron.app/) and include an image here.
#tf.keras.utils.plot_model(model, "model.png", show_shapes=True, show_dtype=True, expand_nested=True) # Uncomment this to generate a simple visualization
from IPython.display import Image
Image('CNN_TinyImageNet.h5.png', width=350)
```

Out[13]:


conv2d_input
?×64×64×3
Conv2D
kernel ⟨5×5×3×32⟩
bias ⟨32⟩
Activation
Conv2D
kernel ⟨5×5×32×32⟩
bias ⟨32⟩
Activation
MaxPooling2D
Conv2D
kernel ⟨3×3×32×64⟩


bias ⟨64⟩
Activation
Conv2D
kernel ⟨3×3×64×64⟩
bias ⟨64⟩
Activation
MaxPooling2D
Conv2D
kernel ⟨3×3×64×64⟩
bias ⟨64⟩
Activation
Conv2D
kernel ⟨3×3×64×128⟩
bias ⟨128⟩
Activation



In [14]:
```
# We can view the layer weights. Here we consider them as images of feature filters applied to intermediate feature map images.
# TODO: Visualize the 2 convolutional layers filter sets (weights) (one at the beginning and one at the end)

# TODO: Your Code Here
```

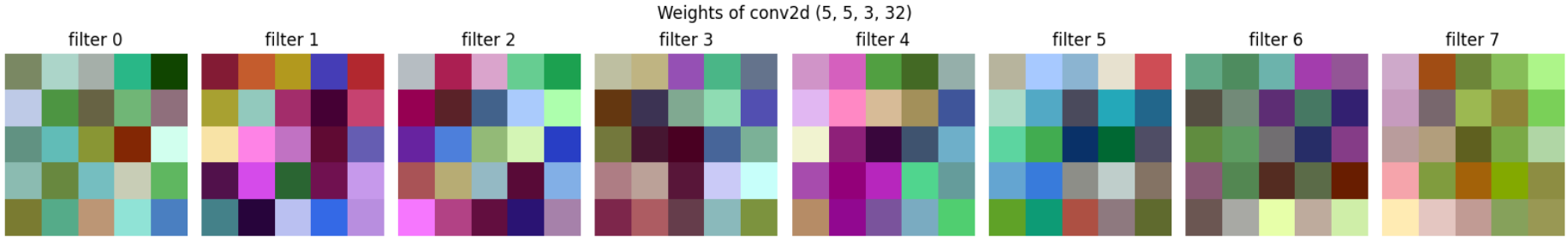
```
conv_layers = [layer for layer in model.layers if isinstance(layer, tf.keras.layers.Conv2D)]
early_layer, late_layer = conv_layers[0], conv_layers[-1]

def show_filters(layer, num_filters=8):
    kernel, bias = layer.get_weights()
    print(f'--- {layer.name} ---')
    print(f'Data type: {kernel.dtype}')
    print(f'Kernel dimensions (H, W, in, out): {kernel.shape}, bias dimensions: {bias.shape}')
    print(f'Memory: {kernel.nbytes} (kernel) + {bias.nbytes} (bias) = {kernel.nbytes + bias.nbytes} bytes')

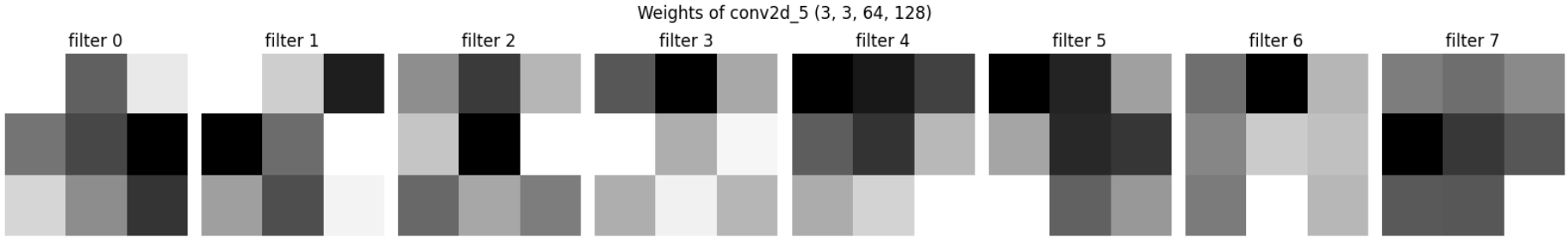
    fig, axes = plt.subplots(1, num_filters, figsize=(2 * num_filters, 2.6))
    for i, ax in enumerate(axes):
        filt = kernel[:, :, :, i]
        if filt.shape[-1] == 3:
            # RGB input: show the whole filter as a color image, rescaled to [0, 1]
            ax.imshow((filt - filt.min()) / (filt.max() - filt.min() + 1e-8))
        else:
            # Many input channels: show the 2D slice that is applied to input channel 0
            ax.imshow(filt[:, :, 0], cmap='gray')
        ax.set_title(f'filter {i}')
        ax.axis('off')
    fig.suptitle(f'Weights of {layer.name} {kernel.shape}')
    plt.tight_layout()
    plt.show()

show_filters(early_layer)
show_filters(late_layer)
```

```
--- conv2d ---
Data type: float32
Kernel dimensions (H, W, in, out): (5, 5, 3, 32), bias dimensions: (32,)
Memory: 9600 (kernel) + 128 (bias) = 9728 bytes
```




```
--- conv2d_5 ---
Data type: float32
Kernel dimensions (H, W, in, out): (3, 3, 64, 128), bias dimensions: (128,)
Memory: 294912 (kernel) + 512 (bias) = 295424 bytes
```



In [15]:
```
# We can view the layer outputs as well. Here we consider them as images of the spatial location of features.
# TODO: Visualize the 2 convolutional layers outputs (intermediate feature maps) (one at the beginning and one at the end)

# TODO: Your Code Here
# One model that returns the output of every layer (reused by the export cell below)
feature_model = tf.keras.Model(inputs=model.inputs, outputs=[layer.output for layer in model.layers])

def run_all_layers(img_data):
    input_tensor = tf.expand_dims(tf.cast(img_data["image"], tf.float32) / 255.0, axis=0)
    outputs = feature_model(input_tensor, training=False)
    return {layer.name: output.numpy() for layer, output in zip(model.layers, outputs)}

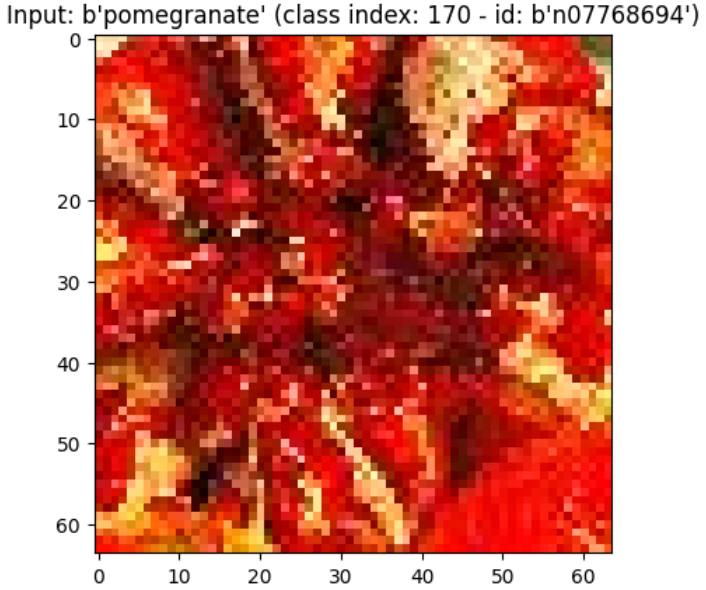
def show_feature_maps(features, layer_name, num_maps=8):
    print(f'--- {layer_name} ---')
    print(f'Data type: {features.dtype}')
    print(f'Feature map dimensions (N, H, W, C): {features.shape}')
    print(f'Memory: {features.nbytes} bytes')

    fig, axes = plt.subplots(1, num_maps, figsize=(2 * num_maps, 2.6))
    for i, ax in enumerate(axes):
        ax.imshow(features[0, :, :, i], cmap='viridis')
        ax.set_title(f'channel {i}')
        ax.axis('off')
    fig.suptitle(f'Feature maps of {layer_name} {features.shape}')
    plt.tight_layout()
    plt.show()

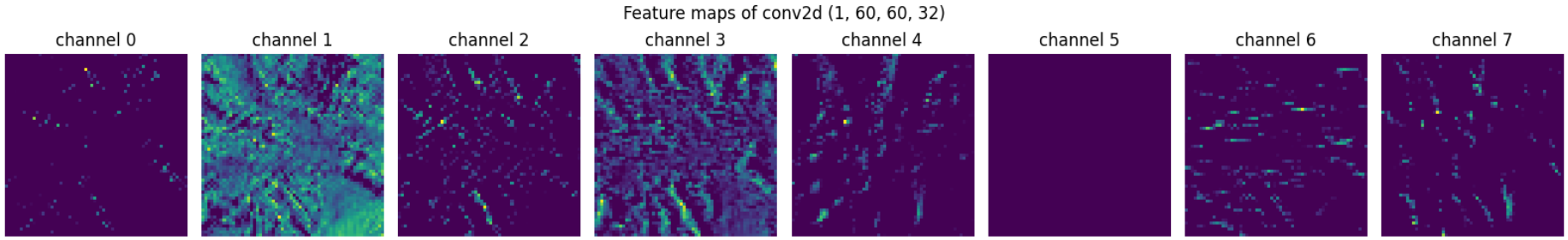
# Use sample image 0 as the test input
img_info(sample_imgs[0], title_apend='Input: ')
plt.show()

sample_outputs = run_all_layers(sample_imgs[0])
show_feature_maps(sample_outputs[early_layer.name], early_layer.name)
show_feature_maps(sample_outputs[late_layer.name], late_layer.name)
```

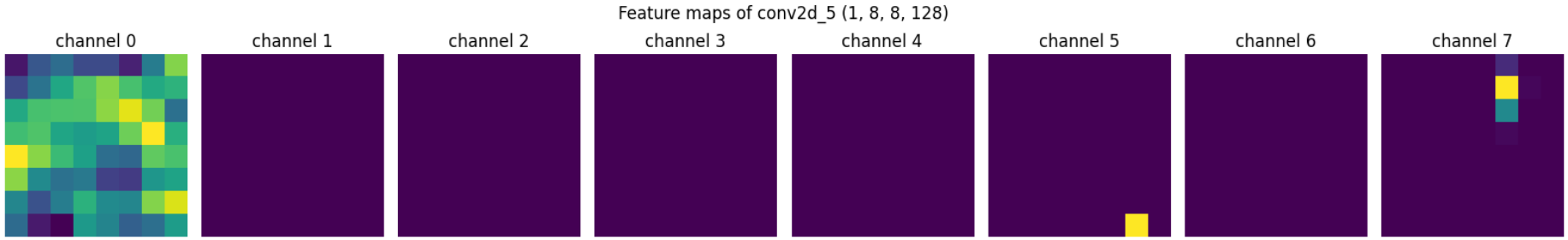
```
Label: b'pomegranate' (class index: 170 - id: b'n07768694')
```



```
--- conv2d ---
Data type: float32
Feature map dimensions (N, H, W, C): (1, 60, 60, 32)
Memory: 460800 bytes
```




```
--- conv2d_5 ---
Data type: float32
Feature map dimensions (N, H, W, C): (1, 8, 8, 128)
Memory: 32768 bytes
```



In [16]:
```python
# TODO: Export the filters/weights se we can use them later
# Make a directory for our image data
model_dir = os.path.abspath('model_data')
pathlib.Path(model_dir).mkdir(exist_ok=True)


# Export each image
conv_index = dense_index = 1 # layer index starts from one
for layer_idx, layer in enumerate(model.layers):
    if re.match(r'(conv|dense)', layer.name):
        weight_file_name = os.path.join(model_dir, f'{layer.name}_weights.bin')
        bias_file_name = os.path.join(model_dir, f'{layer.name}_bias.bin')
    else: continue

    assert layer.weights[0].name.endswith('kernel')
    assert layer.weights[1].name.endswith('bias')

    # TODO: Your Code Here
    kernel, bias = layer.get_weights()

    # Raw float32 in the Keras layout: Conv (H, W, in, out), Dense (in, out)
    kernel.astype(np.float32).tofile(weight_file_name)
    bias.astype(np.float32).tofile(bias_file_name)
    assert os.path.getsize(weight_file_name) == kernel.size * 4
    assert os.path.getsize(bias_file_name) == bias.size * 4

    print(f'{layer.name}: weights {kernel.shape} {kernel.nbytes} bytes, bias {bias.shape} {bias.nbytes} bytes')
```

```
exported_bytes = sum(f.stat().st_size for f in pathlib.Path(model_dir).glob('*.bin'))
print(f'\nExported {exported_bytes} bytes = {exported_bytes // 4} float32 values (model has {model.count_params()} parameters)')
```

```
conv2d: weights (5, 5, 3, 32) 9600 bytes, bias (32,) 128 bytes
conv2d_1: weights (5, 5, 32, 32) 102400 bytes, bias (32,) 128 bytes
conv2d_2: weights (3, 3, 32, 64) 73728 bytes, bias (64,) 256 bytes
conv2d_3: weights (3, 3, 64, 64) 147456 bytes, bias (64,) 256 bytes
conv2d_4: weights (3, 3, 64, 64) 147456 bytes, bias (64,) 256 bytes
conv2d_5: weights (3, 3, 64, 128) 294912 bytes, bias (128,) 512 bytes
dense: weights (2048, 256) 2097152 bytes, bias (256,) 1024 bytes
dense_1: weights (256, 200) 204800 bytes, bias (200,) 800 bytes

Exported 3080864 bytes = 770216 float32 values (model has 770216 parameters)
```

In [17]:

```
# TODO: Export the intermediate layer outputs for each of the input for all of the layers
img_dir = os.path.abspath('img_data')
pathlib.Path(img_dir).mkdir(exist_ok=True)

for img_idx, img in enumerate(sample_imgs):
    file_dir = os.path.join(img_dir, f'test_input_{img_idx}')
    pathlib.Path(file_dir).mkdir(exist_ok=True)

    # TODO: Your Code Here
    # Raw float32, shape (1, H, W, C) for conv/pool layers and (1, N) for flatten/dense
    for layer_name, output in run_all_layers(img).items():
        output.astype(np.float32).tofile(os.path.join(file_dir, f'{layer_name}_output.bin'))
        print(f'Image {img_idx}: {layer_name} {output.shape} {output.nbytes} bytes')

    # The last exported output is the class prediction: it must agree with the inference table above
    print(f'Image {img_idx}: exported prediction = class {int(np.argmax(output))} '
          f'(confidence {output.max():.4f}), actual = class {int(img["label"].numpy())}\n')
```

```
Image 0: conv2d (1, 60, 60, 32) 460800 bytes
Image 0: conv2d_1 (1, 56, 56, 32) 401408 bytes
Image 0: max_pooling2d (1, 28, 28, 32) 100352 bytes
Image 0: conv2d_2 (1, 26, 26, 64) 173056 bytes
Image 0: conv2d_3 (1, 24, 24, 64) 147456 bytes
Image 0: max_pooling2d_1 (1, 12, 12, 64) 36864 bytes
Image 0: conv2d_4 (1, 10, 10, 64) 25600 bytes
Image 0: conv2d_5 (1, 8, 8, 128) 32768 bytes
Image 0: max_pooling2d_2 (1, 4, 4, 128) 8192 bytes
Image 0: flatten (1, 2048) 8192 bytes
Image 0: dense (1, 256) 1024 bytes
Image 0: dense_1 (1, 200) 800 bytes
Image 0: exported prediction = class 170 (confidence 0.7293), actual = class 170

Image 1: conv2d (1, 60, 60, 32) 460800 bytes
Image 1: conv2d_1 (1, 56, 56, 32) 401408 bytes
Image 1: max_pooling2d (1, 28, 28, 32) 100352 bytes
Image 1: conv2d_2 (1, 26, 26, 64) 173056 bytes
Image 1: conv2d_3 (1, 24, 24, 64) 147456 bytes
Image 1: max_pooling2d_1 (1, 12, 12, 64) 36864 bytes
Image 1: conv2d_4 (1, 10, 10, 64) 25600 bytes
Image 1: conv2d_5 (1, 8, 8, 128) 32768 bytes
Image 1: max_pooling2d_2 (1, 4, 4, 128) 8192 bytes
Image 1: flatten (1, 2048) 8192 bytes
Image 1: dense (1, 256) 1024 bytes
Image 1: dense_1 (1, 200) 800 bytes
Image 1: exported prediction = class 106 (confidence 0.1033), actual = class 77

Image 2: conv2d (1, 60, 60, 32) 460800 bytes
Image 2: conv2d_1 (1, 56, 56, 32) 401408 bytes
Image 2: max_pooling2d (1, 28, 28, 32) 100352 bytes
Image 2: conv2d_2 (1, 26, 26, 64) 173056 bytes
Image 2: conv2d_3 (1, 24, 24, 64) 147456 bytes
Image 2: max_pooling2d_1 (1, 12, 12, 64) 36864 bytes
Image 2: conv2d_4 (1, 10, 10, 64) 25600 bytes
Image 2: conv2d_5 (1, 8, 8, 128) 32768 bytes
Image 2: max_pooling2d_2 (1, 4, 4, 128) 8192 bytes
Image 2: flatten (1, 2048) 8192 bytes
Image 2: dense (1, 256) 1024 bytes
Image 2: dense_1 (1, 200) 800 bytes
Image 2: exported prediction = class 128 (confidence 0.2106), actual = class 11
```

Tensorboard

In [5]:
```
# Setup for profiling
tf.profiler.experimental.ProfilerOptions(
    host_tracer_level=1, python_tracer_level=0, device_tracer_level=1
)

log_dir = os.path.abspath(os.path.join('log_data'))
log_dir_run = os.path.abspath(os.path.join(log_dir, datetime.datetime.now().strftime("%Y%m%d-%H%M%S")))
pathlib.Path(log_dir_run).mkdir(exist_ok=True, parents=True)

try:
    tf.profiler.experimental.stop()
except:
    test = 2
finally:
    test = 1
```

In [19]:
```
# TODO: Sample Profiling - Inference for a single image:
import time

def preprocess(images):
    # uint8 [0, 255] -> float32 [0, 1]; a single image also gets the batch dimension
    images = tf.cast(images, tf.float32) / 255.0
    return tf.expand_dims(images, axis=0) if images.shape.ndims == 3 else images

# Warm-up: the first predict call builds the inference graph, which is not part of the inference itself
model.predict(preprocess(sample_imgs[0]["image"]), verbose=0)

# Perform the inference profiling (one profiled inference for each of the 3 sample images):
single_latency_ms = []
# Starts Profile logging
tf.profiler.experimental.start(log_dir_run)
try:
    for index, img_data in enumerate(sample_imgs):
        with tf.profiler.experimental.Trace('single_inference', step_num=index, _r=1):
            # Actual inference
            # TODO: Your Code Here
            start = time.perf_counter()
            model.predict(preprocess(img_data["image"]), verbose=0)
            single_latency_ms.append((time.perf_counter() - start) * 1000)
finally:
    tf.profiler.experimental.stop()

# The profiler adds overhead, so also report the median of 20 un-profiled runs per image
print(f'{"Image":<7}{"Class":<18}{"Profiled (ms)":<15}Median of 20 runs (ms)')
for index, img_data in enumerate(sample_imgs):
    repeats = []
    for _ in range(20):
        start = time.perf_counter()
        model.predict(preprocess(img_data["image"]), verbose=0)
        repeats.append((time.perf_counter() - start) * 1000)
    name = wnid_to_words[wnids[int(img_data["label"].numpy())]][0]
    print(f'{index:<7}{name:<18}{single_latency_ms[index]:<15.2f}{np.median(repeats):.2f}')
```

```
2026-10-07 16:07:54.891886: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:104] Profiler session initializing.
2026-10-07 16:07:54.891917: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:119] Profiler session started.
2026-10-07 16:07:55.126572: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:70] Profiler session collecting data.
2026-10-07 16:07:55.128532: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:131] Profiler session tear down.
2026-10-07 16:07:55.136754: I external/local_tsl/tsl/profiler/rpc/client/save_profile.cc:144] Collecting XSpace to repository: /home/jwk0425/jwk_personal_lab/lab1/log_
data/20261007-160749/plugins/profile/2026_10_07_16_07_55/linuxvdi-f26-19.ece.iastate.edu.xplane.pb
Image  Class             Profiled (ms)  Median of 20 runs (ms)
0      pomegranate       66.32          64.63
1      miniskirt         61.01          57.52
2      German shepherd   100.32         57.29
```

In [20]:
```
# Load the TensorBoard notebook extension.
%load_ext tensorboard

# Launch TensorBoard and navigate to the Profile tab to view performance profile.
# *** Please note just execute this command once in a session and
# then logs for subsequent runs would be auto detected in tensorboard- url: http://localhost:6006/
print(log_dir)
%tensorboard --logdir={log_dir_run} --port=6006

# You can view the tensorboard in the browser url: http://localhost:6006/

# Useful command line to have if tensorboard is misbehaving: kill $(ps -e | grep 'tensorboard' | awk '{print $1}')
```

```
/home/jwk0425/jwk_personal_lab/lab1/log_data
```

Index of file:///

Show hidden objects

Name	Size	Last Modified
File: .autorelabel		8/7/26 13:27:00 CDT
afs		10/2/24 16:00:29 CDT
bin		8/10/26 09:28:41 CDT
boot		8/7/26 13:27:13 CDT
dev		10/7/26 19:36:17 CDT
etc		10/7/26 04:00:29 CDT
home		10/7/26 19:23:45 CDT
lib		8/7/26 14:09:04 CDT
lib64		9/2/26 14:27:28 CDT
local		8/7/26 13:33:30 CDT
lost+found		8/7/26 13:16:41 CDT
media		10/2/24 16:00:29 CDT
mnt		8/7/26 13:21:56 CDT
opt		8/7/26 13:48:29 CDT
proc		10/7/26 04:00:17 CDT
remote		10/7/26 19:32:13 CDT
research		10/7/26 04:00:29 CDT
root		9/9/26 09:47:55 CDT
run		10/7/26 04:00:29 CDT
sbin		8/7/26 14:01:34 CDT
srv		10/2/24 16:00:29 CDT
sys		10/7/26 04:00:17 CDT
tmp		10/7/26 20:53:25 CDT
usr		8/7/26 13:18:56 CDT

In [21]:
```
# Layer-wise computation latency plot (computation time on the y-axis, layer name on the x-axis)
# The values are "Avg. self-time (us)" read from TensorBoard > Profile > framework_op_stats (this is the view the
# lab manual calls tensorflow_stats). Screenshots of the same tables are in the report appendix.
#   online: run 2026_10_07_16_14_51 = 1000 single-image predict calls (1000 occurrences of every operation)
#   batch : run 2026_10_07_16_16_16 = 5 predict calls with batch size 200 (5 occurrences), divided by 200 below
# The operation names carry an extra suffix: layer conv2d is 'sequential_1/conv2d_1/Relu', layer conv2d_1 is
# 'sequential_1/conv2d_1_2/Relu', layer dense is 'dense_1/Relu' and layer dense_1 is 'dense_1_2/BiasAdd' + 'Softmax'.
# flatten does no computation and has no entry. These numbers must be re-read if the profiling cells are re-run.
layer_names = ['conv2d', 'conv2d_1', 'max_pooling2d', 'conv2d_2', 'conv2d_3', 'max_pooling2d_1',
               'conv2d_4', 'conv2d_5', 'max_pooling2d_2', 'dense', 'dense_1']
assert layer_names == [layer.name for layer in model.layers if layer.name != 'flatten']

online_us = np.array([283, 538, 93, 167, 187, 59, 90, 94, 46, 180, 30 + 10])
```

```
# max_pooling2d_2 and dense_1 were below the visible rows; they are the per-type totals minus the visible rows
batch200_us = np.array([8990, 62067, 4213, 12552, 17453, 1252, 3454, 5426, 485, 733, 180 + 59]) / 200

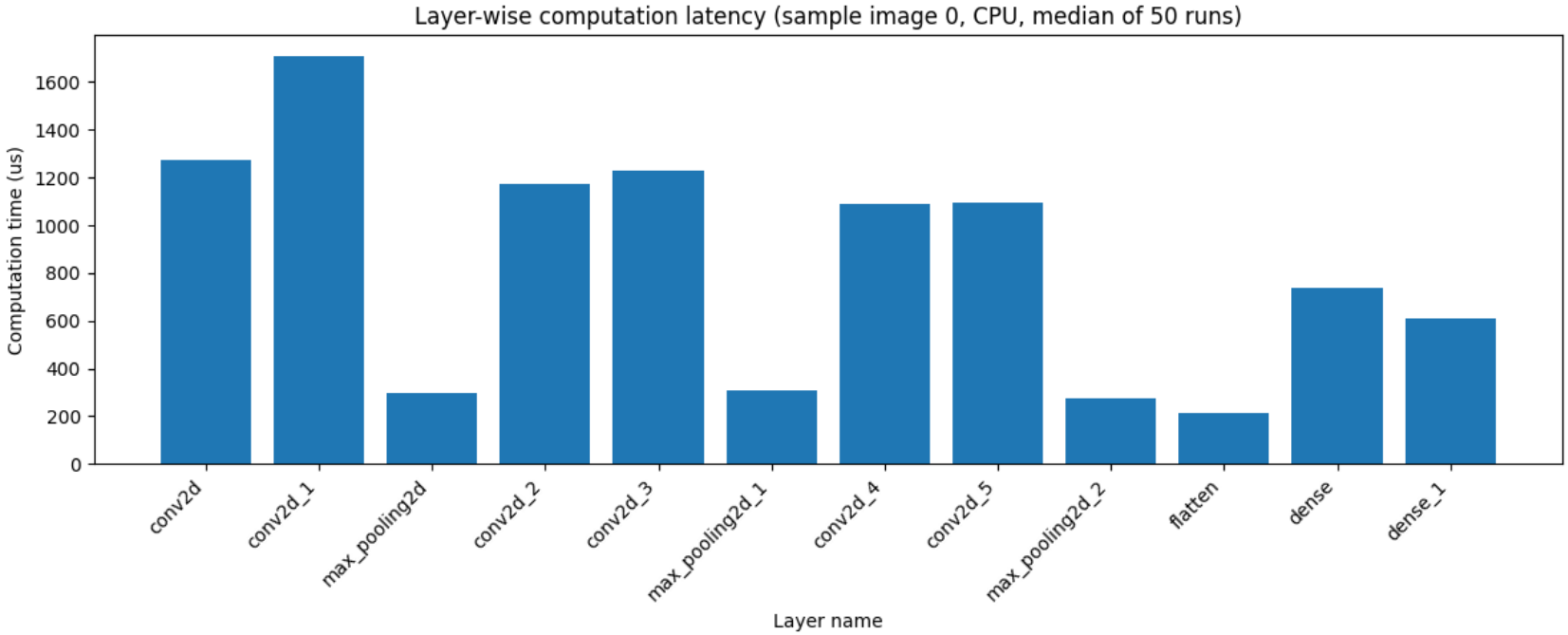
def mac_count(layer):
    # Multiply-accumulate operations needed for one image (0 for pooling layers)
    if not layer.weights:
        return 0
    kernel = layer.get_weights()[0]
    if kernel.ndim == 4:  # conv: every output value needs kernel_h * kernel_w * in_channels MACs
        return int(np.prod(layer.output.shape[1:]) * np.prod(kernel.shape[:3]))
    return int(kernel.size)  # dense: in * out

macs = np.array([mac_count(model.get_layer(name)) for name in layer_names])

print(f'{"Layer":<18}{"online (us)":>12}{"share":>8}{"batch 200 (us)":>16}{"share":>8}{"MACs (M)":>10}')
for name, online, batch, mac in zip(layer_names, online_us, batch200_us, macs):
    print(f'{name:<18}{online:>12.0f}{online / online_us.sum() * 100:>7.1f}%'
          f'{batch:>16.1f}{batch / batch200_us.sum() * 100:>7.1f}%{mac / 1e6:>10.2f}')
print(f'{"total":<18}{online_us.sum():>12.0f}{"":>8}{batch200_us.sum():>16.1f}{"":>8}{macs.sum() / 1e6:>10.2f}')

fig, axes = plt.subplots(1, 2, figsize=(16, 5))
for ax, values, title in ((axes[0], online_us, 'Online inference (batch size 1)'),
                          (axes[1], batch200_us, 'Batch inference (batch size 200)')):
    ax.bar(layer_names, values)
    ax.set_xlabel('Layer name')
    ax.set_ylabel('Computation time per image (us)')
    ax.set_title(f'Layer-wise computation latency, CPU - {title}')
    plt.setp(ax.get_xticklabels(), rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

```
conv2d               1272.9 us   12.7%
conv2d_1             1709.9 us   17.1%
max_pooling2d         299.8 us    3.0%
conv2d_2             1174.6 us   11.7%
conv2d_3             1228.7 us   12.3%
max_pooling2d_1       307.4 us    3.1%
conv2d_4             1087.7 us   10.9%
conv2d_5             1094.4 us   10.9%
max_pooling2d_2       277.9 us    2.8%
flatten               214.8 us    2.1%
dense                 740.0 us    7.4%
dense_1               610.2 us    6.1%
total               10018.3 us
```



In [22]:
```python
# TODO: Sample Profiling - Online Inference:

# The same 1,000 validation images are used for the online and the batch experiments.
# They are loaded into memory first so that reading the dataset is not counted as inference time.
profile_images = tf.stack([example["image"] for example in ds["validation"].take(1000)])

# Vary this from 10, 100, 1000 to simulate multiple online inference
loop_index = [10, 100, 1000]

online_results = {}
for idx in loop_index:
    # Starts Profile logging
    tf.profiler.experimental.start(log_dir_run)

    # Actual online inference
    # TODO: Your Code Here
    try:
        start = time.perf_counter()
        for image in profile_images[:idx]:
            model.predict(preprocess(image), verbose=0)
        latency = time.perf_counter() - start
    finally:
        tf.profiler.experimental.stop()

    online_results[idx] = latency
    print(f'{idx:>5} images: end-to-end latency {latency:8.3f} s, '
          f'throughput {idx / latency:8.2f} images/s, {latency / idx * 1000:7.2f} ms/image')
```

```
2026-10-07 16:13:48.381913: W tensorflow/core/framework/local_rendezvous.cc:404] Local rendezvous is aborting with status: OUT_OF_RANGE: End of sequence
2026-10-07 16:13:48.438985: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:104] Profiler session initializing.
2026-10-07 16:13:48.439035: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:119] Profiler session started.
2026-10-07 16:13:49.042163: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:70] Profiler session collecting data.
2026-10-07 16:13:49.046107: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:131] Profiler session tear down.
2026-10-07 16:13:49.047858: I external/local_tsl/tsl/profiler/rpc/client/save_profile.cc:144] Collecting XSpace to repository: /home/jwk0425/jwk_personal_lab/lab1/log_
data/20261007-160749/plugins/profile/2026_10_07_16_13_49/linuxvdi-f26-19.ece.iastate.edu.xplane.pb
2026-10-07 16:13:49.054111: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:104] Profiler session initializing.
2026-10-07 16:13:49.054134: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:119] Profiler session started.
   10 images: end-to-end latency    0.603 s, throughput    16.58 images/s,   60.30 ms/image
2026-10-07 16:13:55.254968: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:70] Profiler session collecting data.
2026-10-07 16:13:55.284343: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:131] Profiler session tear down.
2026-10-07 16:13:55.285692: I external/local_tsl/tsl/profiler/rpc/client/save_profile.cc:144] Collecting XSpace to repository: /home/jwk0425/jwk_personal_lab/lab1/log_
data/20261007-160749/plugins/profile/2026_10_07_16_13_55/linuxvdi-f26-19.ece.iastate.edu.xplane.pb
2026-10-07 16:13:55.303532: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:104] Profiler session initializing.
2026-10-07 16:13:55.303559: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:119] Profiler session started.
  100 images: end-to-end latency    6.201 s, throughput    16.13 images/s,   62.01 ms/image
2026-10-07 16:14:51.632495: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:70] Profiler session collecting data.
 1000 images: end-to-end latency   56.329 s, throughput    17.75 images/s,   56.33 ms/image
2026-10-07 16:14:51.964121: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:131] Profiler session tear down.
2026-10-07 16:14:51.965695: I external/local_tsl/tsl/profiler/rpc/client/save_profile.cc:144] Collecting XSpace to repository: /home/jwk0425/jwk_personal_lab/lab1/log_
data/20261007-160749/plugins/profile/2026_10_07_16_14_51/linuxvdi-f26-19.ece.iastate.edu.xplane.pb
```

In [23]:

```python
# TODO: Sample Profiling - Batch Inference:

# We would only perform batch inference for a subset of validation set i.e. 1000 images
# using different batch sizes of 20, 40, 100, 200

# Decides the size of the batch. Try: 20, 40, 100, 200
batch_size = [20, 40, 100, 200]

batch_results = {}
for batch in batch_size:
    # Warm-up with this batch shape so that graph building is not counted as latency
    model.predict(preprocess(profile_images[:batch]), batch_size=batch, verbose=0)

    # Starts Profile logging
    tf.profiler.experimental.start(log_dir_run)

    # Actual Batch inference
    # TODO: Your Code Here
    try:
        start = time.perf_counter()
        for i in range(0, len(profile_images), batch):
            # batch_size must be passed explicitly: predict() would otherwise split the input into batches of 32
            model.predict(preprocess(profile_images[i:i + batch]), batch_size=batch, verbose=0)
        latency = time.perf_counter() - start
    finally:
        tf.profiler.experimental.stop()

    batch_results[batch] = latency
    print(f'batch size {batch:>3}: end-to-end latency {latency:8.3f} s, '
          f'throughput {len(profile_images) / latency:8.2f} images/s, {latency / len(profile_images) * 1000:7.2f} ms/image')

print(f'\nonline (batch size 1): end-to-end latency {online_results[1000]:8.3f} s, '
      f'throughput {1000 / online_results[1000]:8.2f} images/s for the same 1000 images')
```

```
2026-10-07 16:16:05.889156: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:104] Profiler session initializing.
2026-10-07 16:16:05.889183: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:119] Profiler session started.
2026-10-07 16:16:10.353782: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:70] Profiler session collecting data.
2026-10-07 16:16:10.381022: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:131] Profiler session tear down.
2026-10-07 16:16:10.382905: I external/local_tsl/tsl/profiler/rpc/client/save_profile.cc:144] Collecting XSpace to repository: /home/jwk0425/jwk_personal_lab/lab1/log_
data/20261007-160749/plugins/profile/2026_10_07_16_16_10/linuxvdi-f26-19.ece.iastate.edu.xplane.pb
2026-10-07 16:16:10.507314: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:104] Profiler session initializing.
2026-10-07 16:16:10.507342: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:119] Profiler session started.
batch size  20: end-to-end latency    4.464 s, throughput   224.01 images/s,    4.46 ms/image
2026-10-07 16:16:13.232906: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:70] Profiler session collecting data.
2026-10-07 16:16:13.250582: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:131] Profiler session tear down.
2026-10-07 16:16:13.252823: I external/local_tsl/tsl/profiler/rpc/client/save_profile.cc:144] Collecting XSpace to repository: /home/jwk0425/jwk_personal_lab/lab1/log_
data/20261007-160749/plugins/profile/2026_10_07_16_16_13/linuxvdi-f26-19.ece.iastate.edu.xplane.pb
2026-10-07 16:16:13.424943: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:104] Profiler session initializing.
2026-10-07 16:16:13.424969: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:119] Profiler session started.
batch size  40: end-to-end latency    2.725 s, throughput   366.91 images/s,    2.73 ms/image
2026-10-07 16:16:15.106118: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:70] Profiler session collecting data.
2026-10-07 16:16:15.112794: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:131] Profiler session tear down.
2026-10-07 16:16:15.115030: I external/local_tsl/tsl/profiler/rpc/client/save_profile.cc:144] Collecting XSpace to repository: /home/jwk0425/jwk_personal_lab/lab1/log_
data/20261007-160749/plugins/profile/2026_10_07_16_16_15/linuxvdi-f26-19.ece.iastate.edu.xplane.pb
batch size 100: end-to-end latency    1.681 s, throughput   594.88 images/s,    1.68 ms/image
2026-10-07 16:16:15.468177: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:104] Profiler session initializing.
2026-10-07 16:16:15.468208: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:119] Profiler session started.
batch size 200: end-to-end latency    1.188 s, throughput   841.62 images/s,    1.19 ms/image

online (batch size 1): end-to-end latency   56.329 s, throughput    17.75 images/s for the same 1000 images
2026-10-07 16:16:16.656759: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:70] Profiler session collecting data.
2026-10-07 16:16:16.659897: I external/local_tsl/tsl/profiler/lib/profiler_session.cc:131] Profiler session tear down.
2026-10-07 16:16:16.662655: I external/local_tsl/tsl/profiler/rpc/client/save_profile.cc:144] Collecting XSpace to repository: /home/jwk0425/jwk_personal_lab/lab1/log_
data/20261007-160749/plugins/profile/2026_10_07_16_16_16/linuxvdi-f26-19.ece.iastate.edu.xplane.pb
```

Training

In [6]:
```python
# Setup for model training
from tensorflow.keras import Model, datasets
from tensorflow.keras.models import Sequential
from tensorflow.keras.losses import categorical_crossentropy
from tensorflow.keras.optimizers import SGD
from tensorflow.keras.layers import Dense, Flatten, Conv2D, AveragePooling2D, MaxPooling2D, ZeroPadding2D,Convolution2D, Activation, Dropout

train_dir = os.path.abspath(os.path.join('train_data', datetime.datetime.now().strftime("%Y%m%d-%H%M%S")))
pathlib.Path(train_dir).mkdir(exist_ok=True, parents=True)

# Using early stopping to monitor validation accuracy
callbacks = [
    tf.keras.callbacks.EarlyStopping(
        # Stop training when `val_loss` is no longer improving
        monitor="val_loss",
        # "no longer improving" being defined as "no better than 1e-2 less"
        min_delta=1e-2,
        # "no longer improving" being further defined as "for at least 2 epochs"
        patience=2,
        verbose=1,
    ),
    tf.keras.callbacks.TensorBoard(log_dir=train_dir, histogram_freq=1)
]
```

In [7]:
```python
# Basic CNN model
train_model = Sequential()
```

```
# conv1
train_model.add(Conv2D(32, (5, 5), input_shape=(64, 64, 3), activation='relu'))
train_model.add(Conv2D(32, (5,5),activation='relu'))
train_model.add(MaxPooling2D(pool_size=(2, 2)))
train_model.add(Conv2D(64, (3,3), activation='relu'))
train_model.add(Conv2D(64, (3,3), activation='relu'))
train_model.add(MaxPooling2D(pool_size=(2, 2)))
train_model.add(Conv2D(64, (3,3), activation='relu'))
train_model.add(Conv2D(128, (3,3), activation='relu'))
train_model.add(MaxPooling2D(pool_size=(2, 2)))
train_model.add(Flatten())

# fc1
train_model.add(Dense(256, activation='relu'))

# fc2
train_model.add(Dense(200, activation='softmax'))

train_model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
# TODO: Consider looking at different optimizers and learning rate settings
train_model.summary()
```

```
/home/jwk0425/jwk_personal_lab/lab1/lab1_venv/lib64/python3.9/site-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/
`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 60, 60, 32)	2,432
conv2d_1 (Conv2D)	(None, 56, 56, 32)	25,632
max_pooling2d (MaxPooling2D)	(None, 28, 28, 32)	0
conv2d_2 (Conv2D)	(None, 26, 26, 64)	18,496
conv2d_3 (Conv2D)	(None, 24, 24, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 12, 12, 64)	0
conv2d_4 (Conv2D)	(None, 10, 10, 64)	36,928
conv2d_5 (Conv2D)	(None, 8, 8, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 4, 4, 128)	0
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 256)	524,544
dense_1 (Dense)	(None, 200)	51,400

Total params: 770,216 (2.94 MB)

Trainable params: 770,216 (2.94 MB)

Non-trainable params: 0 (0.00 B)

In [8]:
```
# TODO: Attempt to train your own model with different batch sizes
# TODO: See how long this takes without a GPU on your VDI or 2050 Coover machines
```

```
# TODO: THEN log in to your GPU VM, set ENABLE_GPU = False in the very first cell, and re-run all above cells
# TODO: Make sure you have exported the LD_LIBRARY_PATH as the lab manual indicates

def normalize_img(image, label):
    return tf.cast(image, tf.float32) / 255., label

def to_categorical(image, label):
    label = tf.one_hot(tf.cast(label, tf.int32), 200)
    return tf.cast(image, tf.float32), tf.cast(label, tf.int64)

ds_re = tiny_imagenet_builder.as_dataset(as_supervised=True)
ds_retrain, ds_reval = ds_re["train"], ds_re["validation"]

ds_retrain = ds_retrain.cache().shuffle(1024)
ds_reval = ds_reval.cache().shuffle(1024)

ds_retrain = ds_retrain.map(normalize_img, num_parallel_calls=tf.data.AUTOTUNE)
ds_reval = ds_reval.map(normalize_img, num_parallel_calls=tf.data.AUTOTUNE)

ds_retrain = ds_retrain.map(to_categorical, num_parallel_calls=tf.data.AUTOTUNE)
ds_reval = ds_reval.map(to_categorical, num_parallel_calls=tf.data.AUTOTUNE)

# Adam with the default learning rate (1e-3) sometimes collapsed in our earlier runs: the loss froze at
# ln(200) = 5.2983, i.e. the model predicted the same uniform distribution for every image. A lower learning rate
# is used here, and train_once() prints a warning if a run ends in that collapsed state.
import time
LEARNING_RATE = 3e-4
UNIFORM_LOSS = float(np.log(200))

def train_once(batch_size, epochs):
    run_name = f'batch{batch_size}_epoch{epochs}'
    tf.keras.utils.set_random_seed(487)  # same initial weights and shuffling for every run

    # Setup our batched datasets
    train_batch = ds_retrain.batch(batch_size).prefetch(tf.data.AUTOTUNE)
    val_batch = ds_reval.batch(batch_size).prefetch(tf.data.AUTOTUNE)

    # A fresh, randomly initialized copy of the model, so that every run starts from scratch
    model_run = tf.keras.models.clone_model(train_model)
    model_run.compile(loss='categorical_crossentropy',
                      optimizer=tf.keras.optimizers.Adam(learning_rate=LEARNING_RATE),
                      metrics=[tf.keras.metrics.CategoricalAccuracy(name='top1'),
                               tf.keras.metrics.TopKCategoricalAccuracy(k=5, name='top5')])

    # The EarlyStopping callback from the setup cell, plus a TensorBoard log directory per run
    run_callbacks = [callbacks[0], tf.keras.callbacks.TensorBoard(log_dir=os.path.join(train_dir, run_name))]

    print(f'\n===== {run_name} =====')
    start = time.perf_counter()
    history = model_run.fit(train_batch, validation_data=val_batch, epochs=epochs,
                            callbacks=run_callbacks, verbose=2).history
    elapsed = time.perf_counter() - start

    best = int(np.argmax(history['val_top1']))
    result = {'batch': batch_size, 'epochs': epochs, 'ran': len(history['loss']), 'best_epoch': best + 1,
              'val_top1': history['val_top1'][best], 'val_top5': max(history['val_top5']),
              'last_val_top1': history['val_top1'][-1], 'train_top1': history['top1'][-1],
              'sec_per_epoch': elapsed / len(history['loss'])}
    if history['loss'][-1] > 0.98 * UNIFORM_LOSS:
        print(f'WARNING: training collapsed (loss {history["loss"][-1]:.4f} ~ ln(200) = {UNIFORM_LOSS:.4f}); '
              f'this run did not learn and must not be used for conclusions')
    return model_run, result
```

```python
def print_results(results):
    print(f'\n{"batch":>6}{"epochs":>8}{"ran":>5}{"best epoch":>12}{"best val top-1":>16}{"best val top-5":>16}'
          f'{"last val top-1":>16}{"last train top-1":>18}{"s/epoch":>9}')
    for r in results:
        print(f'{r["batch"]:>6}{r["epochs"]:>8}{r["ran"]:>5}{r["best_epoch"]:>12}{r["val_top1"] * 100:>15.2f}%'
              f'{r["val_top5"] * 100:>15.2f}%{r["last_val_top1"] * 100:>15.2f}%{r["train_top1"] * 100:>17.2f}%'
              f'{r["sec_per_epoch"]:>9.1f}')

epoch_size = 20

batch_size_results = []
for batch_size in [32, 64, 128]:
    # Setup our batched datasets
    # TODO: Your Code Here
    _, result = train_once(batch_size, epoch_size)
    batch_size_results.append(result)

print_results(batch_size_results)
```

```
===== batch32_epoch20 =====
Epoch 1/20
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1791409536.361966  532442 service.cc:145] XLA service 0x7f176000e3f0 initialized for platform CUDA (this does not guarantee that XLA will be used). Device
s:
I0000 00:00:1791409536.362007  532442 service.cc:153]   StreamExecutor device (0): NVIDIA A40-8Q, Compute Capability 8.6
2026-10-07 16:45:36.424933: I tensorflow/compiler/mlir/tensorflow/utils/dump_mlir_util.cc:268] disabling MLIR crash reproducer, set env var `MLIR_CRASH_REPRODUCER_DIRE
CTORY` to enable.
2026-10-07 16:45:36.686106: I external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:465] Loaded cuDNN version 8907
I0000 00:00:1791409540.859449  532442 device_compiler.h:188] Compiled cluster using XLA!  This line is logged at most once for the lifetime of the process.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1791409556.762959  532720 asm_compiler.cc:369] ptxas warning : Registers are spilled to local memory in function 'triton_gemm_dot_163', 8 bytes spill store
s, 8 bytes spill loads

3125/3125 - 22s - 7ms/step - loss: 4.8760 - top1: 0.0394 - top5: 0.1280 - val_loss: 4.4515 - val_top1: 0.0794 - val_top5: 0.2332
Epoch 2/20
3125/3125 - 15s - 5ms/step - loss: 4.1192 - top1: 0.1274 - top5: 0.3202 - val_loss: 3.9851 - val_top1: 0.1462 - val_top5: 0.3549
Epoch 3/20
3125/3125 - 15s - 5ms/step - loss: 3.7222 - top1: 0.1858 - top5: 0.4169 - val_loss: 3.7023 - val_top1: 0.1850 - val_top5: 0.4148
Epoch 4/20
3125/3125 - 15s - 5ms/step - loss: 3.4632 - top1: 0.2272 - top5: 0.4745 - val_loss: 3.5641 - val_top1: 0.2136 - val_top5: 0.4566
Epoch 5/20
3125/3125 - 15s - 5ms/step - loss: 3.2574 - top1: 0.2629 - top5: 0.5214 - val_loss: 3.5124 - val_top1: 0.2244 - val_top5: 0.4697
Epoch 6/20
3125/3125 - 15s - 5ms/step - loss: 3.0782 - top1: 0.2944 - top5: 0.5605 - val_loss: 3.4278 - val_top1: 0.2440 - val_top5: 0.4863
Epoch 7/20
3125/3125 - 15s - 5ms/step - loss: 2.9120 - top1: 0.3247 - top5: 0.5951 - val_loss: 3.4663 - val_top1: 0.2410 - val_top5: 0.4837
Epoch 8/20
3125/3125 - 15s - 5ms/step - loss: 2.7602 - top1: 0.3525 - top5: 0.6238 - val_loss: 3.4831 - val_top1: 0.2419 - val_top5: 0.4892
Epoch 8: early stopping

===== batch64_epoch20 =====
Epoch 1/20
```

```
I0000 00:00:1791409664.134224  533079 asm_compiler.cc:369] ptxas warning : Registers are spilled to local memory in function 'triton_gemm_dot_746', 312 bytes spill sto
res, 280 bytes spill loads

I0000 00:00:1791409664.302722  533073 asm_compiler.cc:369] ptxas warning : Registers are spilled to local memory in function 'triton_gemm_dot_746', 88 bytes spill stor
es, 88 bytes spill loads

I0000 00:00:1791409664.403479  533076 asm_compiler.cc:369] ptxas warning : Registers are spilled to local memory in function 'triton_gemm_dot_901', 112 bytes spill sto
res, 112 bytes spill loads

I0000 00:00:1791409664.521540  533074 asm_compiler.cc:369] ptxas warning : Registers are spilled to local memory in function 'triton_gemm_dot_901', 112 bytes spill sto
res, 112 bytes spill loads

1563/1563 - 18s - 12ms/step - loss: 4.9408 - top1: 0.0325 - top5: 0.1135 - val_loss: 4.5075 - val_top1: 0.0772 - val_top5: 0.2261
Epoch 2/20
1563/1563 - 11s - 7ms/step - loss: 4.2198 - top1: 0.1138 - top5: 0.2981 - val_loss: 4.0048 - val_top1: 0.1395 - val_top5: 0.3441
Epoch 3/20
1563/1563 - 11s - 7ms/step - loss: 3.7967 - top1: 0.1741 - top5: 0.4012 - val_loss: 3.7220 - val_top1: 0.1841 - val_top5: 0.4150
Epoch 4/20
1563/1563 - 11s - 7ms/step - loss: 3.5432 - top1: 0.2156 - top5: 0.4581 - val_loss: 3.5776 - val_top1: 0.2080 - val_top5: 0.4462
Epoch 5/20
1563/1563 - 11s - 7ms/step - loss: 3.3453 - top1: 0.2490 - top5: 0.5012 - val_loss: 3.4957 - val_top1: 0.2260 - val_top5: 0.4671
Epoch 6/20
1563/1563 - 11s - 7ms/step - loss: 3.1819 - top1: 0.2777 - top5: 0.5370 - val_loss: 3.4231 - val_top1: 0.2392 - val_top5: 0.4814
Epoch 7/20
1563/1563 - 11s - 7ms/step - loss: 3.0310 - top1: 0.3041 - top5: 0.5688 - val_loss: 3.4285 - val_top1: 0.2426 - val_top5: 0.4843
Epoch 8/20
1563/1563 - 11s - 7ms/step - loss: 2.8953 - top1: 0.3290 - top5: 0.5970 - val_loss: 3.4479 - val_top1: 0.2499 - val_top5: 0.4928
Epoch 8: early stopping

===== batch128_epoch20 =====
Epoch 1/20
I0000 00:00:1791409759.025333  533527 asm_compiler.cc:369] ptxas warning : Registers are spilled to local memory in function 'triton_gemm_dot_746', 252 bytes spill sto
res, 220 bytes spill loads

I0000 00:00:1791409759.533356  533528 asm_compiler.cc:369] ptxas warning : Registers are spilled to local memory in function 'triton_gemm_dot_901', 116 bytes spill sto
res, 116 bytes spill loads

I0000 00:00:1791409759.594929  533532 asm_compiler.cc:369] ptxas warning : Registers are spilled to local memory in function 'triton_gemm_dot_746', 88 bytes spill stor
es, 88 bytes spill loads

I0000 00:00:1791409760.223373  533529 asm_compiler.cc:369] ptxas warning : Registers are spilled to local memory in function 'triton_gemm_dot_901', 8 bytes spill store
s, 8 bytes spill loads
```

```
782/782 - 17s - 22ms/step - loss: 4.9432 - top1: 0.0343 - top5: 0.1151 - val_loss: 4.6754 - val_top1: 0.0664 - val_top5: 0.1841
Epoch 2/20
782/782 - 8s - 11ms/step - loss: 4.3984 - top1: 0.0960 - top5: 0.2546 - val_loss: 4.3018 - val_top1: 0.1114 - val_top5: 0.2780
Epoch 3/20
782/782 - 8s - 11ms/step - loss: 4.0091 - top1: 0.1483 - top5: 0.3518 - val_loss: 3.9412 - val_top1: 0.1556 - val_top5: 0.3648
Epoch 4/20
782/782 - 8s - 11ms/step - loss: 3.7344 - top1: 0.1892 - top5: 0.4167 - val_loss: 3.7567 - val_top1: 0.1851 - val_top5: 0.4112
Epoch 5/20
782/782 - 8s - 11ms/step - loss: 3.5426 - top1: 0.2202 - top5: 0.4612 - val_loss: 3.6237 - val_top1: 0.2113 - val_top5: 0.4401
Epoch 6/20
782/782 - 8s - 11ms/step - loss: 3.3786 - top1: 0.2485 - top5: 0.4967 - val_loss: 3.5815 - val_top1: 0.2204 - val_top5: 0.4537
Epoch 7/20
782/782 - 8s - 11ms/step - loss: 3.2363 - top1: 0.2709 - top5: 0.5275 - val_loss: 3.5735 - val_top1: 0.2235 - val_top5: 0.4575
Epoch 8/20
782/782 - 8s - 11ms/step - loss: 3.1077 - top1: 0.2939 - top5: 0.5537 - val_loss: 3.5104 - val_top1: 0.2336 - val_top5: 0.4728
Epoch 9/20
782/782 - 8s - 11ms/step - loss: 2.9903 - top1: 0.3137 - top5: 0.5771 - val_loss: 3.5236 - val_top1: 0.2369 - val_top5: 0.4806
Epoch 10/20
782/782 - 8s - 11ms/step - loss: 2.8773 - top1: 0.3361 - top5: 0.5994 - val_loss: 3.4924 - val_top1: 0.2482 - val_top5: 0.4857
Epoch 11/20
782/782 - 8s - 11ms/step - loss: 2.7754 - top1: 0.3540 - top5: 0.6216 - val_loss: 3.4996 - val_top1: 0.2463 - val_top5: 0.4846
Epoch 12/20
782/782 - 8s - 11ms/step - loss: 2.6709 - top1: 0.3731 - top5: 0.6432 - val_loss: 3.5316 - val_top1: 0.2481 - val_top5: 0.4892
Epoch 12: early stopping
```

batch	epochs	ran	best epoch	best val top-1	best val top-5	last val top-1	last train top-1	s/epoch
32	20	8	6	24.40%	48.92%	24.19%	35.25%	15.9
64	20	8	8	24.99%	49.28%	24.99%	32.90%	11.8
128	20	12	10	24.82%	48.92%	24.81%	37.31%	9.1

In [9]:
```python
# TODO: Train your model with 3 different numbers of epochs
batch_size = 32

# Setup your datasets
# TODO: Your Code Here
# (train_once() from the cell above batches ds_retrain / ds_reval with this batch size)

epoch_results = []
for epoch_size in [3, 10, 100]:
    # Run training
    # TODO: Your Code Here
    trained_model, result = train_once(batch_size, epoch_size)
    epoch_results.append(result)

    # Save the cnn model (the number of epochs is part of the file name so that the runs do not overwrite each other)
    trained_model.save(os.path.join(log_dir, f'CNN_TinyImageNet_train_batch{batch_size}_epoch{epoch_size}.h5'))

    # TODO: Get the top-1 and top-5 of your newly trained model
    # TODO: Your Code Here
    _, top1, top5 = trained_model.evaluate(ds_reval.batch(batch_size), verbose=0)
    print(f'{epoch_size} epochs requested, {result["ran"]} run: '
          f'final model validation Top-1 {top1 * 100:.2f}%, Top-5 {top5 * 100:.2f}%')

print_results(epoch_results)
```

```
===== batch32_epoch3 =====
Epoch 1/3
3125/3125 - 18s - 6ms/step - loss: 4.8764 - top1: 0.0381 - top5: 0.1284 - val_loss: 4.4999 - val_top1: 0.0734 - val_top5: 0.2204
Epoch 2/3
3125/3125 - 15s - 5ms/step - loss: 4.1808 - top1: 0.1182 - top5: 0.3047 - val_loss: 3.9162 - val_top1: 0.1488 - val_top5: 0.3637
Epoch 3/3
3125/3125 - 15s - 5ms/step - loss: 3.7728 - top1: 0.1748 - top5: 0.4047 - val_loss: 3.6764 - val_top1: 0.1893 - val_top5: 0.4296
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend usin
g instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
3 epochs requested, 3 run: final model validation Top-1 18.93%, Top-5 42.96%

===== batch32_epoch10 =====
Epoch 1/10
3125/3125 - 18s - 6ms/step - loss: 4.8819 - top1: 0.0376 - top5: 0.1263 - val_loss: 4.5493 - val_top1: 0.0725 - val_top5: 0.2065
Epoch 2/10
3125/3125 - 15s - 5ms/step - loss: 4.2313 - top1: 0.1115 - top5: 0.2930 - val_loss: 4.0660 - val_top1: 0.1298 - val_top5: 0.3322
Epoch 3/10
3125/3125 - 15s - 5ms/step - loss: 3.8194 - top1: 0.1704 - top5: 0.3922 - val_loss: 3.7492 - val_top1: 0.1777 - val_top5: 0.4049
Epoch 4/10
3125/3125 - 15s - 5ms/step - loss: 3.5544 - top1: 0.2139 - top5: 0.4549 - val_loss: 3.5897 - val_top1: 0.2060 - val_top5: 0.4454
Epoch 5/10
3125/3125 - 15s - 5ms/step - loss: 3.3397 - top1: 0.2498 - top5: 0.5018 - val_loss: 3.5277 - val_top1: 0.2193 - val_top5: 0.4573
Epoch 6/10
3125/3125 - 15s - 5ms/step - loss: 3.1543 - top1: 0.2817 - top5: 0.5420 - val_loss: 3.4795 - val_top1: 0.2295 - val_top5: 0.4758
Epoch 7/10
3125/3125 - 15s - 5ms/step - loss: 2.9822 - top1: 0.3118 - top5: 0.5769 - val_loss: 3.5042 - val_top1: 0.2313 - val_top5: 0.4774
Epoch 8/10
3125/3125 - 15s - 5ms/step - loss: 2.8238 - top1: 0.3400 - top5: 0.6112 - val_loss: 3.4370 - val_top1: 0.2432 - val_top5: 0.4898
Epoch 9/10
3125/3125 - 15s - 5ms/step - loss: 2.6689 - top1: 0.3672 - top5: 0.6418 - val_loss: 3.5104 - val_top1: 0.2441 - val_top5: 0.4850
Epoch 10/10
3125/3125 - 15s - 5ms/step - loss: 2.5213 - top1: 0.3948 - top5: 0.6710 - val_loss: 3.6420 - val_top1: 0.2400 - val_top5: 0.4783
Epoch 10: early stopping
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend usin
g instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
10 epochs requested, 10 run: final model validation Top-1 24.00%, Top-5 47.83%

===== batch32_epoch100 =====
Epoch 1/100
3125/3125 - 18s - 6ms/step - loss: 4.8638 - top1: 0.0399 - top5: 0.1323 - val_loss: 4.4370 - val_top1: 0.0813 - val_top5: 0.2403
Epoch 2/100
3125/3125 - 15s - 5ms/step - loss: 4.1449 - top1: 0.1222 - top5: 0.3155 - val_loss: 3.9584 - val_top1: 0.1445 - val_top5: 0.3520
Epoch 3/100
3125/3125 - 15s - 5ms/step - loss: 3.7381 - top1: 0.1819 - top5: 0.4135 - val_loss: 3.6740 - val_top1: 0.1895 - val_top5: 0.4236
Epoch 4/100
3125/3125 - 15s - 5ms/step - loss: 3.4819 - top1: 0.2232 - top5: 0.4700 - val_loss: 3.5778 - val_top1: 0.2066 - val_top5: 0.4470
Epoch 5/100
3125/3125 - 15s - 5ms/step - loss: 3.2820 - top1: 0.2582 - top5: 0.5149 - val_loss: 3.5059 - val_top1: 0.2236 - val_top5: 0.4702
Epoch 6/100
3125/3125 - 15s - 5ms/step - loss: 3.1000 - top1: 0.2884 - top5: 0.5541 - val_loss: 3.4491 - val_top1: 0.2320 - val_top5: 0.4841
Epoch 7/100
3125/3125 - 15s - 5ms/step - loss: 2.9381 - top1: 0.3179 - top5: 0.5882 - val_loss: 3.4884 - val_top1: 0.2302 - val_top5: 0.4834
Epoch 8/100
3125/3125 - 15s - 5ms/step - loss: 2.7847 - top1: 0.3458 - top5: 0.6179 - val_loss: 3.4454 - val_top1: 0.2386 - val_top5: 0.4952
Epoch 8: early stopping
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend usin
g instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
100 epochs requested, 8 run: final model validation Top-1 23.86%, Top-5 49.52%

 batch  epochs  ran  best epoch  best val top-1  best val top-5  last val top-1  last train top-1  s/epoch
    32       3    3           3          18.93%          42.96%          18.93%            17.48%     15.9
    32      10   10           9          24.41%          48.98%          24.00%            39.48%     15.2
    32     100    8           8          23.86%          49.52%          23.86%            34.58%     15.2
```

Above and Beyond

In [ ]:
```
# Benchmark our dataset to make sure loading our data isn't a bottleneck ... and because we can
# (This can be skipped since it can take a bit and is't all that important)

# tfds.benchmark(ds_train.batch(32), batch_size=32, num_iter=2**20)
```

In [ ]:
```
# Explore new models to find a higher-accuracy model. Does the new model require more or less time?
```